

Seqlock(序列锁)

一个为读优化的快照原语,以及为何它的正确性迫使我们使用栅栏——在 acquire/release 够不到的地方

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要。当一个单写者需要把一份频繁更新、可能很大的快照——一份行情盘口、一团配置数据——发布给一大群对延迟极度敏感的读者时,seqlock 是首选的数据结构。本报告不把最终代码当作既成事实直接抛出;它把整个设计重建为一连串的张力。我们先说明为什么几个显而易见的工具(读写锁、`std::atomic<BigStruct>`、MPMC 队列)各自失败,从而逼出 seqlock 的核心反转:队列预防数据竞争,而 seqlock 容忍类竞争的读,再事后检测、失败重试。接着我们直面最尖锐的张力——朴素的 seqlock 按 C++ 内存模型的字面定义是一次数据竞争、因而是未定义行为,尽管它在实践中完美工作——并予以化解。最后我们推导内存序,并发现普通的 acquire/release 在此处结构性地不够用:因为被保护的载荷是非原子的,没有任何原子操作可供单向栅栏“绑定”,正确性转而依赖独立的栅栏(fence)从两侧把载荷访问“夹住”。写者的 release fence 与读者的 acquire fence 构成精确的对偶。

1. 问题,以及三个失败的工具

这个场景极度不对称。一个写者以疯狂的速率更新一份快照——比如当前的盘口顶档(买价、卖价、挂单量、时间戳),几十个字节。几十个策略线程以更疯狂的速率读它。读远多于写,而读路径必须尽可能接近“免费”:无锁、无原子写、不阻塞。在动手造任何新东西之前,我们该先问:为什么现成的工具不行?

1.1 读写锁——读者之间互相阻塞

一把 `shared_mutex` 允许多个读者同时持锁,听起来对路。但每个读者为了获取和释放锁,仍要对锁状态做一次原子读-改-写,争抢同一条缓存行;读者在这条线上串行化,并让它在多核间反复弹跳。更糟的是,写者必须等所有读者退出,读者也必须等写者。在一条每秒被穿越数百万次的热读路径上,每次读都付一次有争用的原子操作——并偶尔阻塞——正是我们付不起的代价。

1.2 `std::atomic<BigStruct>`——悄悄退化成一把锁

如果快照足够小,你本可以把它做成单个 `std::atomic` 来无锁地读。但 `std::atomic<T>` 只有当 `T` 能装进平台原生原子宽度时才无锁——x86 上是 8 字节,带 `cmpxchg16b` 时 16 字节。一份 64 字节的盘口装不下。于是编译器在 `atomic<BigStruct>` 内部偷偷藏了一把锁,`is_lock_free()` 返回 `false`。我们又回到了 1.1,读者在不知不觉中拿了一把互斥锁。这个失败正是 seqlock 存在的全部理由:它让一个大到无法做 lock-free atomic 的结构,获得无锁、不阻塞的读——办法是把原子性从载荷上挪走,挪到一个仅 8 字节的计数器上。

1.3 MPMC 队列——语义完全不对

一提“并发数据结构”,本能反应就是无锁队列,但它回答的是另一个问题。队列建模的是一条流:每个元素都必须被消费,不能丢,且被一个消费者 pop 走的元素对其他消费者就不存在了。而行情读者要的恰恰相反——最

新的快照,旧值全部丢弃。用队列的话,写者会堆积读者根本不想要的历史 tick;想要最新值的读者得先把积压排空,而 pop 语义意味着两个读者无法同时拿到同一个最新值。seqlock 则直接在原地覆盖同一份快照;N 个读者同时读同一份当前值,没有积压、没有消费。流 vs 快照正是分水岭,而行情数据牢牢落在“快照”一侧。

第 1 节的裁决。读写锁有争用、会阻塞;atomic<BigStruct> 退化成一一把隐藏的锁;MPMC 队列建模的是流而非快照。这个缺口——一份大的、非原子的快照,却仍要支持无锁、不阻塞、多读者的访问——正是 seqlock 的生态位。

2. 核心反转:容忍并检测,而非预防

SPSC 环形缓冲区(另一份报告的主题)靠预防取得正确性:一条 happens-before 边保证读者绝不触碰写者仍在写的槽位。seqlock 付不起预防的代价——锁住载荷正是我们排除掉的东西——于是它把哲学反转。它让读者乐观地往前走,可能读到一份撕裂的、更新到一半的快照,然后事后检测这次读是否干净,不干净就重试。预防被替换为“检测-重试”。

与 SPSC 的切割。SPSC 用 happens-before 预防竞争,使读者永不触碰争用中的槽位。seqlock 则固有地包含一次类竞争的读:读者必须先把(可能脏的)载荷读出来,才能比对版本号决定丢弃与否。因此 seqlock 面对的是一个 SPSC 根本不存在的问题——这不是炒冷饭,而是相反哲学被推到极致后逼出的新张力(见第 3 节)。

检测器是一个单调递增的序列计数器 seq_,只有一条不变量:写入进行中时它是奇数,数据静止时它是偶数。

- 写者:把 seq_ 加到奇数(“我在写”),更新载荷,再把 seq_ 加到偶数(“写完了”)。
- 读者:把 seq_ 快照为 s1,读载荷,再把 seq_ 快照为 s2。仅当 s1 == s2 且 s1 为偶数时接受这次读;否则丢弃重试。

读者的两个检查把职责切得很干净,值得分别说明,因为它们极易被混为一谈。s1 为偶数意味着开始读时没有写入在进行——若它是奇数,说明写者正在更新半途,这次读必然作废,于是连载荷都不必相信、直接重试。s1 ==

s2意味着读的整个过程中没有写入完成过——若写者在我们读的期间跑完了一整轮,计数器至少前进 2,相等性即不成立。两者同时成立,就意味着整次载荷读完整地落在两次写之间的某个静默窗口里,故这份快照是内部一致的。

一个常见的混淆,予以澄清。seqlock 的载荷从不为空——它始终持有当前快照。因此读者从不像队列消费者等待空队列那样去“等数据到来”。重试循环等的不是数据,而是一次干净的读。s1 == s2(且偶数)是退出条件——这次读干净,拿走走人。不等或为奇数是继续条件——快照在读的中途被扰动,丢掉它,立即再读一份(总有一份可读)。

3. 最尖锐的张力:实践中正确,标准中未定义

设计在这里撞上它最令人不安的真相,而这正是一个严肃的面试官会紧追的问题。读者读非原子的 data_ 时,写者可能正在写同一个 data_,二者之间无锁、无 happens-before 边。按为环形缓冲区所确立的同一判据,这就是一次数据竞争:两个线程、同一非原子对象、至少一个是写、无定序。而 C++ 中的数据竞争就是未定义行为。

那个令人不安的事实。一个朴素的 seqlock——读者对普通非原子的 `data_` 直接做 `result = data_;`——按 C++11 的字面定义是一次数据竞争,因而是 UB,尽管它在每一个真实编译器与 CPU 上都完美工作。重试救不了它:读这个动作已经与写并发地发生了;事后丢弃它的结果无法撤销已经构成的 UB。未定义行为在读发生的那一刻就已招致,而非在使用结果的那一刻。

对此有三个层次的诚实回答,一个强工程师应当同时持有这三层。

第一层——实践派。它到处都能跑。读者观察到的任何撕裂或半写字节,都会被 `seq_` 比对捕获并丢弃,绝不外泄进程序逻辑;在硬件层面,载荷的 `load` 与 `store` 就是普通内存访问,不会“爆炸”。Linux 内核以这种方式用了 seqlock 几十年,HFT 系统每天依赖它。

第二层——符合标准的修补。要让它标准意义上站得住脚,载荷在并发期间就不能被当作普通非原子对象访问。现代修法是通过 relaxed 原子操作访问 `data_`——逐字段原子,或 C++20 的 `atomic_ref`。原子访问即便是 relaxed 也永不构成数据竞争,故 UB 消失;而因为是 relaxed,它不施加任何顺序,在 x86 上也只是一条普通 `mov`,零额外开销。真正的顺序完全由 `seq_` 承载。

第三层——栅栏。对载荷用 relaxed 原子去掉了竞争,但不施加任何顺序;顺序——即保证载荷访问被夹在两次序列读之间——由栅栏提供,这是第 4 节的主题。

把反转重述为哲学。SPSC 用一条 happens-before 边预防竞争,使读者绝不触碰争用中的槽位。seqlock 则容忍一次类竞争的访问,并用版本计数器事后检测它。一个预防;另一个检测并重试。HFT 两者都用:预防代价低时就预防(SPSC);预防太贵时——一份读远多于写的大快照——就检测重试(seqlock)。

4. 内存序:为什么 acquire/release 不够用

人们很容易假设 seqlock 的定序方式和环形缓冲区一样——store 配 release、load 配 acquire,对称成对。这个假设在两处破裂,而追究为何破裂,正是整个设计的核心。

4.1 写者一侧

```
void store(const T& desired) {           // single writer
    uint64_t s = seq_.load(relaxed);
    seq_.store(s + 1, /* [1] */);         // enter write (odd)
    /* fence? */
    data_ = desired;                     // write payload
    seq_.store(s + 2, /* [2] */);         // done (even)
}
```

[2] 是一个 release store,这个简单。它的职责是阻止前面的载荷写下沉到它之后,从而让观察到偶数 seq_ 的读者确信载荷已完整写入。release——它把前面的写焊在 store 之上——恰好做这件事。无需栅栏。

[1] 才是陷阱。出于对称的本能,人们会写 seq_.store(s+1, release),但那个 release 是空转的:[1] 之前没有任何东西要发布——载荷是在 [1] 之后才写的,而非之前。[1] 真正需要的是相反的方向:它必须阻止后面的载荷写向上浮到“我在写”这个宣告之前,这样读者就不会在载荷已被改动时却看到一个偶数(静默)计数器。release 不约束那个方向——release

把前面的写向下焊住,它不阻止后面的写向上浮。正确的工具是一道独立的 release fence,放在 [1] 之后、载荷写之前:

```
seq_.store(s + 1, relaxed);               // [1] announce (relaxed is fine)
std::atomic_thread_fence(memory_order_release); // block payload write from rising
data_ = desired;
seq_.store(s + 2, release);               // [2] welds payload write down
```

4.2 读者一侧——对偶

```
T load() const {           // multiple readers
    T result; uint64_t s1, s2;
    do {
        s1 = seq_.load(/* [3] */);
        result = data_;           // read payload
        /* fence? */
        s2 = seq_.load(/* [4] */);
    } while (s1 != s2 || (s1 & 1));
    return result;
}
```

读者必须把 result = data_ 夹在两次序列读之间——若载荷读漂到第一次之上、或第二次之下,那么 s1 与 s2 就不再框住读的真实时刻,版本检查随之失效。

[3] 是一个 acquire load。读到 s1 之后,载荷读不能被提到它之上。acquire——它阻止后面的读向上浮过该 load——恰好做这件事。

[4] 是一个 acquire fence,而非 acquire load。这正是写者那个陷阱的对偶。[4] 处的约束是:载荷读不能下沉到第二次序列读之下——而 acquire load 约束的是错误的方向(acquire 阻止上浮,而非下沉)。正确的工具是一道独立的 acquire fence,放在载荷读之后、第二次序列 load 之前;s2 本身于是可以是一个 relaxed load,因为栅栏已经锚住了顺序:

```
s1 = seq_.load(acquire);           // [3] block payload read from rising
result = data_;
std::atomic_thread_fence(memory_order_acquire); // block payload read from sinking
s2 = seq_.load(relaxed);           // value only; fence did the ordering
```

4.3 更深的原因:没有可供绑定栅栏的对象

这个反复出现的意外——写者用 release fence、读者用 acquire fence,而人们本以为是普通的 release/release——有一个单一的根源。acquire 或 release 是绑定在某个具体原子操作上的单向栅栏;它约束的是“那个操作”与“它之前或之后的访问”。而 fence 是横在两条语句之间、不绑定任何东西的双侧之墙,约束的是一切跨越它的访问。seqlock 必须给 data_ 的访问定序,但 data_ 不是一个原子操作——它是对一大块非原子载荷的普通(或 relaxed)访问。载荷上没有任何原子操作可供 acquire 或 release 附着。无物可绑,能给载荷立栅栏的唯一工具就是独立的 fence,在两侧立起来把它夹住。

可移植的规则。要给一个本身就是原子的访问定序(环形缓冲区的 tail/head 加载与存储),把 acquire/release 直接绑在它上面——单向就够。要给一个非原子的访问定序(seqlock 的载荷,或任何“一个小原子旗子守护一大团非原子数据”的场景),没有可绑的对象,就用独立的 fence 在要紧的那一侧把它挡住。写者与读者的栅栏构成对偶:写者的 release fence 阻止载荷写上浮过“正在写”,读者的 acquire fence 阻止载荷读下沉到第二次检查之下。这个模式——一个原子旗子、一大块非原子载荷——在 RCU、双缓冲、以及任何大对象发布中反复出现;seqlock 是它的典范实例。

4.4 平台笔记

在 x86-TSO 上,那个朴素的对称猜测(写者两次 store 都 release、读者两次 load 都 acquire、没有显式栅栏)恰好能正确运行,因为硬件本就禁止 store-store 与 load-load 重排——而那正是栅栏所守护的方向。因此这些栅栏在 x86 上几乎免费(常常编译为空),却在 ARM 这类弱序架构上不可或缺——在那里载荷访问真的会漂出夹层。把栅栏显式写出来,才能让 seqlock 在所有平台上都正确,而不只是在你恰好测试的那台机器上。

5. 综述

seqlock

凭借拒绝每一个常规工具、并反转通常的契约,赢得了它的位置。在锁会阻塞读者、atomic<BigStruct> 会偷偷变成一把锁的地方,seqlock 把原子性挪到一个 8 字节计数器上、让大载荷保持非原子,从而让读路径无锁且不阻塞。在队列建模一条流的地方,seqlock 建模一份快照。在环形缓冲区预防竞争的地方,seqlock 容忍并检测竞争。而在 acquire/release 足以应付一个原子旗子的地方,非原子的载荷却没有任何东西可供单向栅栏绑定,于是正确性落到一对对偶的栅栏上,把载荷从两侧夹住——写者一道 release fence,读者一道 acquire fence。这个结构很小;但使它正确的推理,跨越了从缓存争用到 C++ 抽象机的整段距离——而这,正是它值得被推导、而非仅仅被抄写的原因。